

21CSC201J – DATA STRUCTURES AND ALGORITHMS

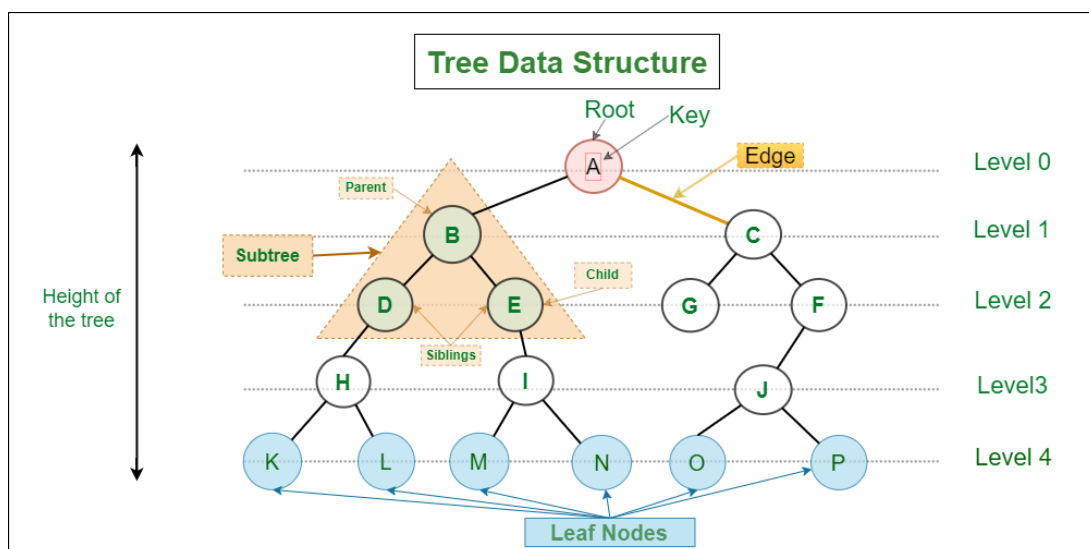
Unit-4

Trees and Hashing

Introduction to Trees, Tree traversals, Complete Binary Tree and its height, Binary Search Trees, Need for Balance, Rotation, AVL trees, B Trees, Heaps, trees and array implementations and applications; Hash functions - Introduction, functions, Collision avoidance, Separate chaining, Open Addressing, Linear Probing, Quadratic probing.

INTRODUCTION TO TREES

- A tree is recursively defined as a set of one or more nodes where one node is designated as the root of the tree and all the remaining nodes can be partitioned into non-empty sets each of which is a sub-tree of the root.



- Figure below shows a tree where node A is the root node; nodes B, C, and D are children of the root node and form sub-trees of the tree rooted at node A.

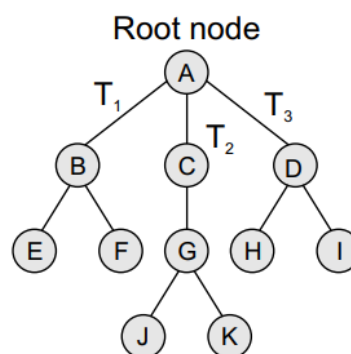


Figure: Tree

Basic Terminology

Root node

- The root node R is the topmost node in the tree. If $R = \text{NULL}$, then it means the tree is empty.

Sub-trees

- If the root node R is not NULL, then the trees T1, T2, and T3 are called the sub-trees of R.

Leaf node

- A node that has no children is called the leaf node or the terminal node.

Path

- A sequence of consecutive edges is called a path. For example, in Fig. 9.1, the path from the root node A to node I is given as: A, D, and I.

Ancestor node

- An ancestor of a node is any predecessor node on the path from root to that node. The root node does not have any ancestors. Nodes A, C, and G are the ancestors of node K.

Descendant node

- A descendant node is any successor node on any path from the node to a leaf node. Leaf nodes do not have any descendants. In the tree, nodes C, G, J, and K are the descendants of node A.

Level number

- Every node in the tree is assigned a level number in such a way that the root node is at level 0, children of the root node are at level number 1. Thus, every node is at one level higher than its parent. So, all child nodes have a level number given by parent's level number + 1.

Degree

- Degree of a node is equal to the number of children that a node has. The degree of a leaf node is zero.

In-degree

- In-degree of a node is the number of edges arriving at that node.

Out-degree

- Out-degree of a node is the number of edges leaving that node.

Binary Trees

- A binary tree is a data structure that is defined as a collection of elements called nodes. In a binary tree, the topmost element is called the root node, and each node has 0, 1, or at the most 2 children.
- A node that has zero children is called a leaf node or a terminal node. Every node contains a data element, a left pointer which points to the left child, and a right pointer which points to the right child.
- The root element is pointed by a 'root' pointer. If root = NULL, then it means the tree is empty.

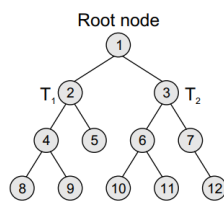


Figure : Binary tree

- In the above figure, R is the root node and the two trees T1 and T2 are called the left and right sub-trees of R. T1 is said to be the left successor of R. Likewise, T2 is called the right successor of R.

Level number

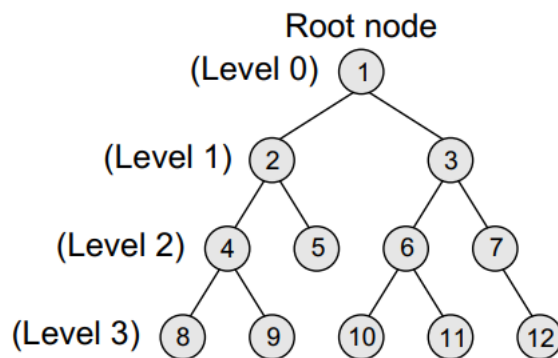


Figure : Levels in binary tree

- Every node in the binary tree is assigned a level number. The root node is defined to be at level 0. The left and the right child of the root node have a level number 1. Similarly, every node is at one level higher than its parents. So all child nodes are defined to have level number as parent's level number + 1.

Degree of a node

- It is equal to the number of children that a node has. The degree of a leaf node is zero.

EXPRESSION TREES

- Binary trees are widely used to store algebraic expressions. For example, consider the algebraic expression given as: $\text{Exp} = (a - b) + (c * d)$.
- This expression can be represented using a binary tree as shown in Figure below.

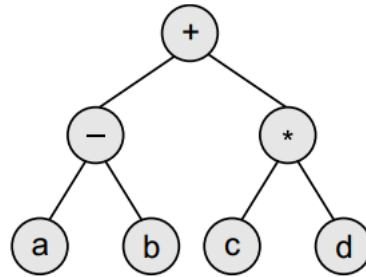


Figure : Expression tree

- Given an expression, $\text{Exp} = ((a + b) - (c * d)) \% ((e \wedge f) / (g - h))$, construct the corresponding binary tree.

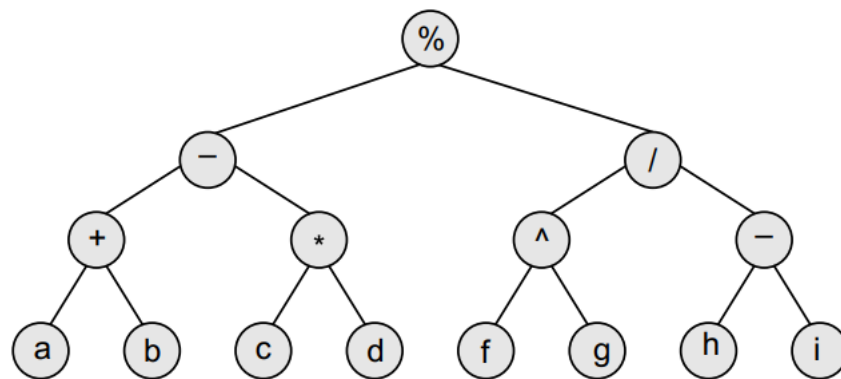


Figure : Expression tree

TREE TRAVERSALS

- Traversing a binary tree is the process of visiting each node in the tree exactly once in a systematic way. Unlike linear data structures in which the elements are traversed sequentially, tree is a nonlinear data structure in which the elements can be traversed in many different ways.
- There are different algorithms for tree traversals. These algorithms differ in the order in which the nodes are visited.

Pre-order Traversal

- To traverse a non-empty binary tree in pre-order, the following operations are performed recursively at each node. The algorithm works by:

1. Visiting the root node,
2. Traversing the left sub-tree, and finally
3. Traversing the right sub-tree.

- The pre-order traversal of the tree is given as A, B, C. Root node first, the left sub-tree next, and then the right sub-tree. Pre-order traversal is also called as depth-first

traversal. In this algorithm, the left sub-tree is always traversed before the right sub-tree.

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:     Write TREE -> DATA
Step 3:     PREORDER(TREE -> LEFT)
Step 4:     PREORDER(TREE -> RIGHT)
           [END OF LOOP]
Step 5: END
```

Figure: Algorithm for pre-order traversal

Example:

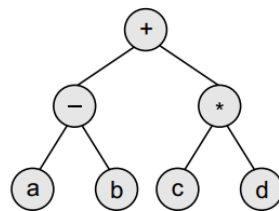
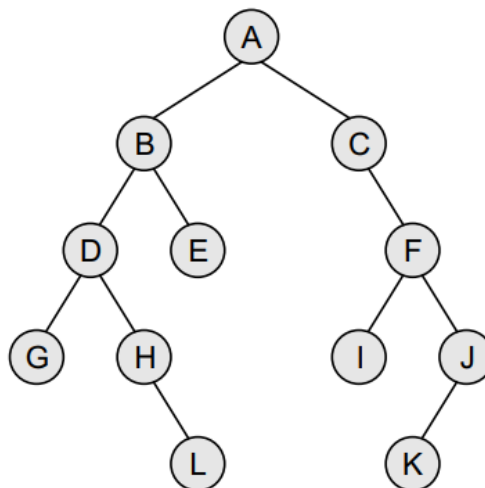


Figure: Expression tree

- When we traverse the elements of a tree using the pre-order traversal algorithm, the expression that we get is a prefix expression **+ - a b * c d**



PRE-ORDER TRAVERSAL ORDER: A, B, D, G, H, L, E, C, F, I, J, and K

In-order Traversal

- To traverse a non-empty binary tree in in-order, the following operations are performed recursively at each node. The algorithm works by:

1. Traversing the left sub-tree,

2. Visiting the root node, and finally

3. Traversing the right sub-tree.

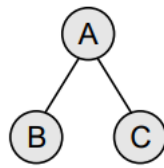


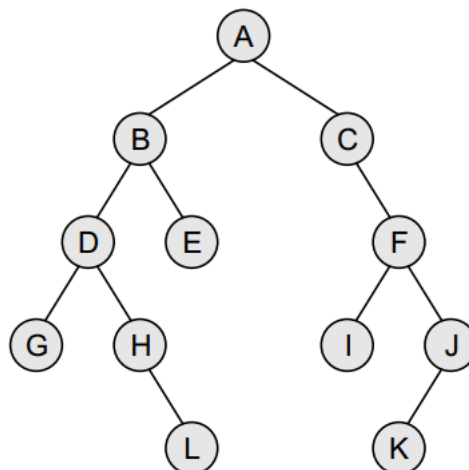
Figure: Binary tree

- The in-order traversal of the tree is given as B, A, and C. Left sub-tree first, the root node next, and then the right sub-tree. In-order traversal is also called as symmetric traversal.

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:         INORDER(TREE -> LEFT)
Step 3:         Write TREE -> DATA
Step 4:         INORDER(TREE -> RIGHT)
              [END OF LOOP]
Step 5: END
```

Figure: Algorithm for in-order traversal

Example:



In-order traversal order : G, D, H, L, B, E, A, C, I, F, K, and J

Post-order Traversal

- To traverse a non-empty binary tree in post-order, the following operations are performed recursively at each node. The algorithm works by:
 1. Traversing the left sub-tree,
 2. Traversing the right sub-tree, and finally
 3. Visiting the root node.
- The post-order traversal of the tree is given as B, C, and A. Left sub-tree first, the right sub-tree next, and finally the root node.

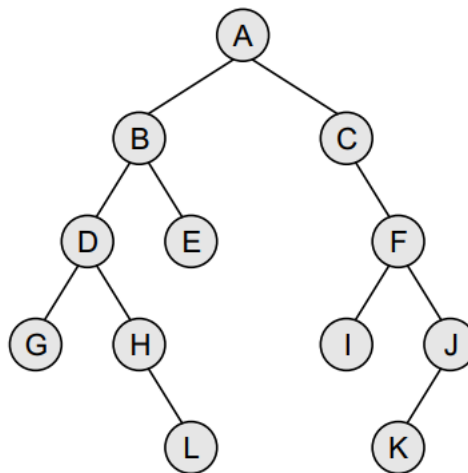
```

Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:         POSTORDER(TREE -> LEFT)
Step 3:         POSTORDER(TREE -> RIGHT)
Step 4:         Write TREE -> DATA
               [END OF LOOP]
Step 5: END

```

Figure : Algorithm for post-order traversal

Example:

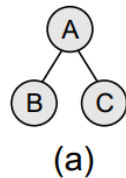


POST-ORDER TRAVERSAL: G, L, H, D, E, B, I, K, J, F, C, and A

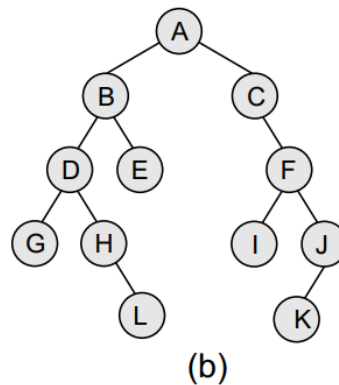
Level-order Traversal

In level-order traversal, all the nodes at a level are accessed before going to the next level. This algorithm is also called as the breadth-first traversal algorithm.

Example:



TRAVERSAL ORDER:
A, B, and C



TRAVERSAL ORDER:
A, B, C, D, E, F, G, H, I, J, L, and K

COMPLETE BINARY TREES

- A complete binary tree is a binary tree that satisfies two properties. First, in a complete binary tree, every level, except possibly the last, is completely filled. Second, all nodes appear as far left as possible.
- In a complete binary tree T_n , there are exactly n nodes and level r of T can have at most 2^r nodes. Figure below shows a complete binary tree.

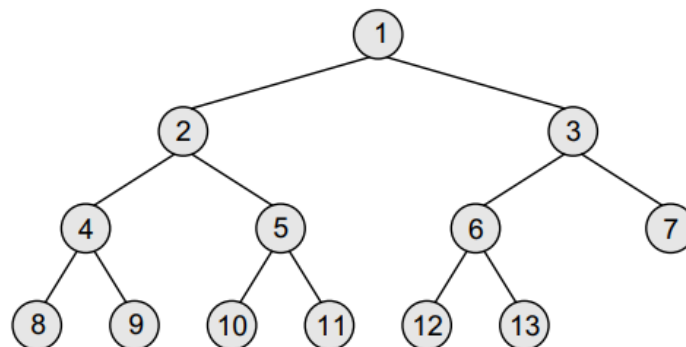


Figure : Complete binary tree

- The height of a tree T_n having exactly n nodes is given as: $H_n = \lceil \log_2(n + 1) \rceil$
- This means, if a tree T has 10,00,000 nodes, then its height is 21.

BINARY SEARCH TREES

- In a binary search tree, all the nodes in the left sub-tree have a value less than that of the root node.
- Correspondingly, all the nodes in the right sub-tree have a value either equal to or greater than the root node. The same rule is applicable to every sub-tree in the tree.
- The root node is 39. The left sub-tree of the root node consists of nodes 9, 10, 18, 19, 21, 27, 28, 29, and 36. All these nodes have smaller values than the root node. The right sub-tree of the root node consists of nodes 40, 45, 54, 59, 60, and 65.

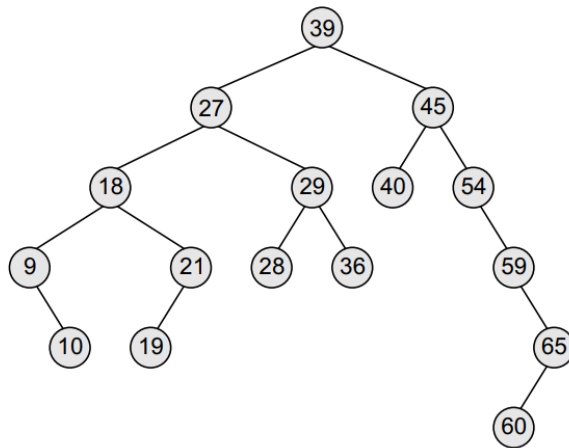


Figure : Binary search tree

- Binary search trees are considered to be efficient data structures especially when compared with sorted linear arrays and linked lists.
- In a sorted array, searching can be done in $O(\log_2 n)$ time, but insertions and deletions are quite expensive. In contrast, inserting and deleting elements in a linked list is easier, but searching for an element is done in $O(n)$ time.
- However, in the worst case, a binary search tree will take $O(n)$ time to search for an element. The worst case would occur when the tree is a linear chain of nodes (Left skewed or right skewed)

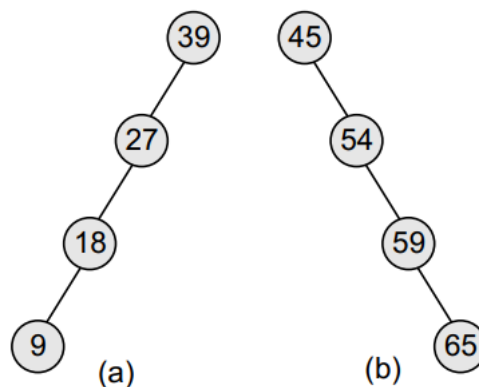
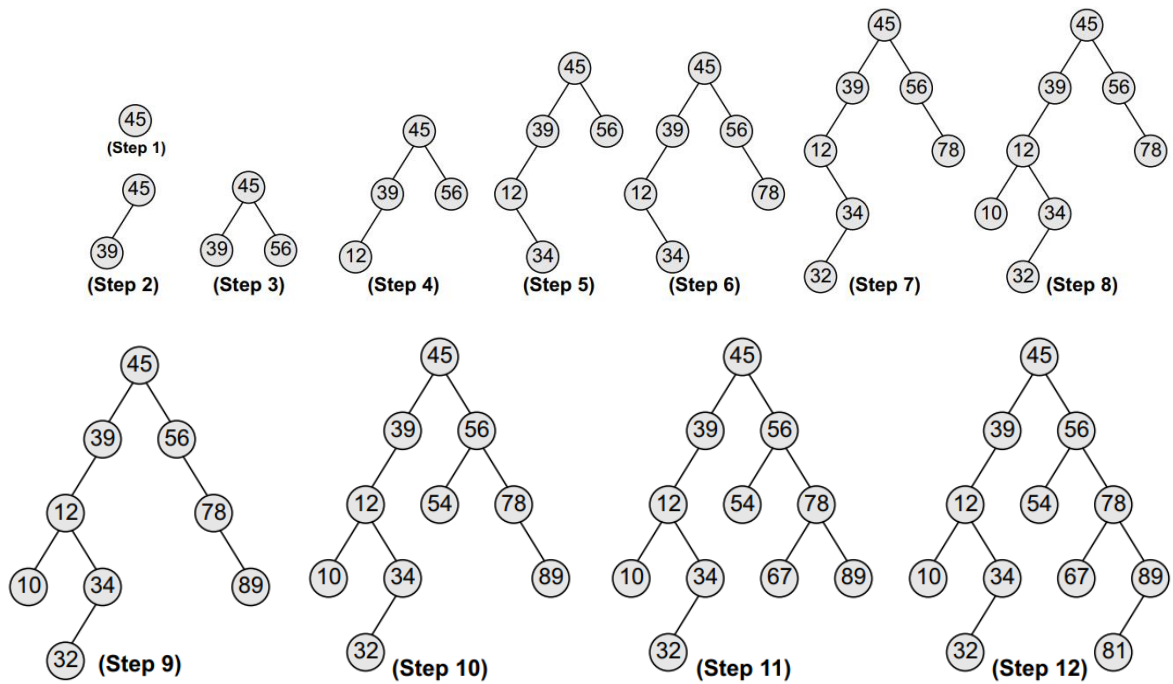


Figure : (a) Left skewed, and (b) right skewed binary search trees

Example:

Create a binary search tree using the following data elements: 45, 39, 56, 12, 34, 78, 32, 10, 89, 54, 67, 81

Solution:



Searching for a Node in a Binary Search Tree

- The search function is used to find whether a given value is present in the tree or not. The **searching process begins at the root node**.
- The function first checks if the binary search tree is empty. If it is empty, then the value we are searching for is not present in the tree.
- If there are nodes in the tree, then the search function checks to see if the key value of the current node is equal to the value to be searched.
- If not, it checks if the value to be searched for is less than the value of the current node, in which case it should be recursively called on the left child node.
- In case the value is greater than the value of the current node, it should be recursively called on the right child node.

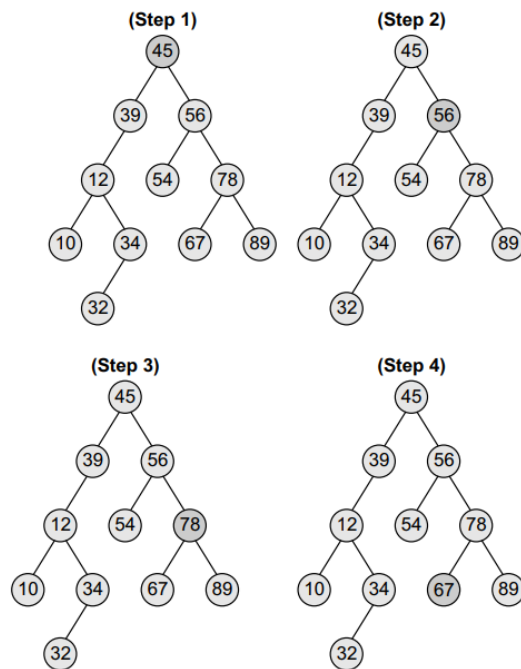


Figure : Searching a node with value 67 in the given binary search tree

ALGORITHM:

```

searchElement (TREE, VAL)

Step 1: IF TREE -> DATA = VAL OR TREE = NULL
    Return TREE
ELSE
    IF VAL < TREE -> DATA
        Return searchElement(TREE -> LEFT, VAL)
    ELSE
        Return searchElement(TREE -> RIGHT, VAL)
    [END OF IF]
    [END OF IF]
Step 2: END
  
```

Inserting a New Node in a Binary Search Tree

- The insert function is used to add a new node with a given value at the correct position in the binary search tree.
- Adding the node at the correct position means that the new node should not violate the properties of the binary search tree.

ALGORITHM:

- In Step 1 of the algorithm, the insert function checks if the current node of TREE is NULL. If it is NULL, the algorithm simply adds the node, else it looks at the current node's value and then recurs down the left or right sub-tree.

```

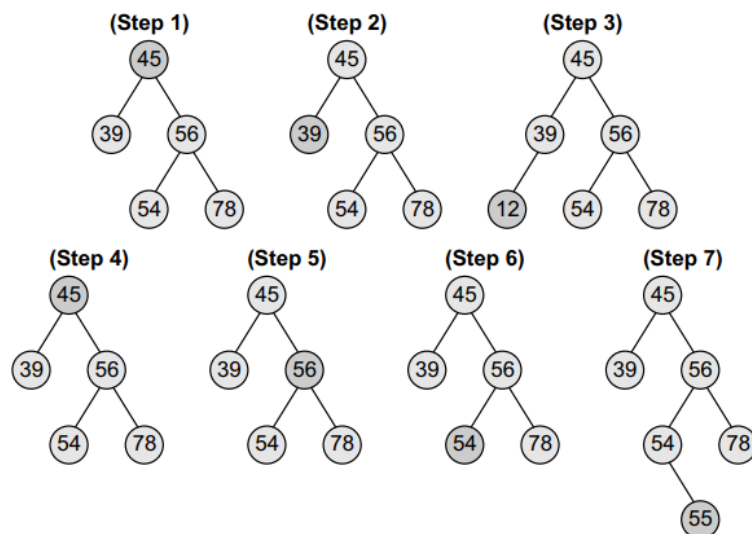
Insert (TREE, VAL)

Step 1: IF TREE = NULL
        Allocate memory for TREE
        SET TREE->DATA = VAL
        SET TREE->LEFT = TREE->RIGHT = NULL
    ELSE
        IF VAL < TREE->DATA
            Insert(TREE->LEFT, VAL)
        ELSE
            Insert(TREE->RIGHT, VAL)
        [END OF IF]
    [END OF IF]
Step 2: END

```

- If the current node's value is less than that of the new node, then the right sub-tree is traversed, else the left sub-tree is traversed. The insert function continues moving down the levels of a binary tree until it reaches a leaf node.
- The new node is added by following the rules of the binary search trees. That is, if the new node's value is greater than that of the parent node, the new node is inserted in the right sub-tree, else it is inserted in the left sub-tree.
- The insert function requires time proportional to the height of the tree in the worst case. It takes $O(\log n)$ time to execute in the average case and $O(n)$ time in the worst case.

Example: Inserting nodes with values 12 and 55 in the given binary search tree



Deleting a Node from a Binary Search Tree

- There are three cases:
 - Case 1: Deleting a Node that has No Children
 - Case 2: Deleting a Node with One Child
 - Case 3: Deleting a Node with Two Children

Case 1: Deleting a Node that has No Children

- If we have to delete node 78, we can simply remove this node without any issue. This is the simplest case of deletion.

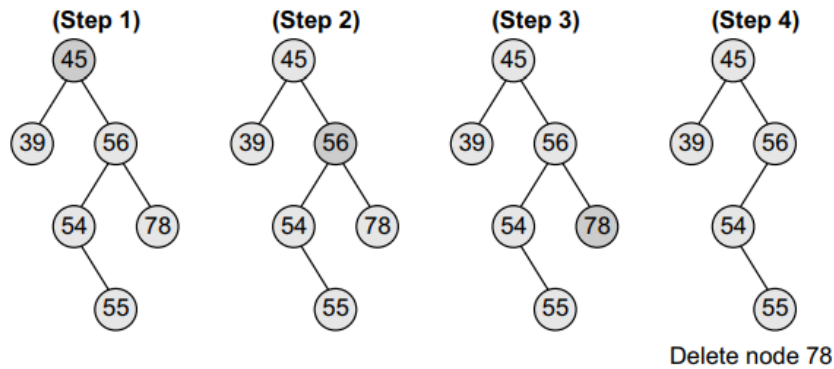


Figure : Deleting node 78 from the given binary search tree

Case 2: Deleting a Node with One Child

- To handle this case, the node's child is set as the child of the node's parent. In other words, replace the node with its child.
- Now, if the node is the left child of its parent, the node's child becomes the left child of the node's parent.
- Correspondingly, if the node is the right child of its parent, the node's child becomes the right child of the node's parent.

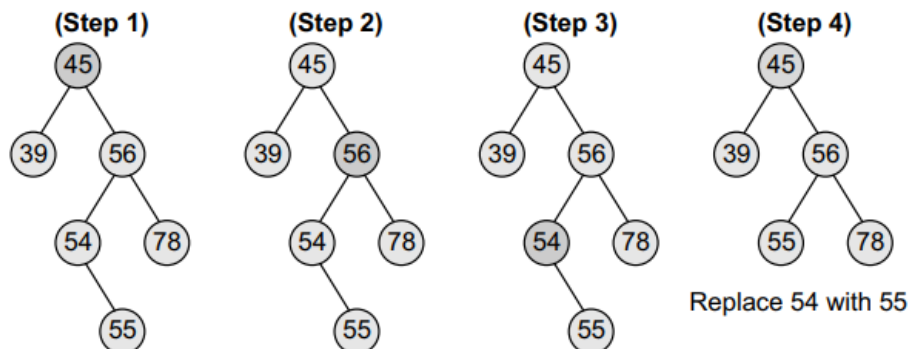


Figure : Deleting node 54 from the given binary search tree

Case 3: Deleting a Node with Two Children

- To handle this case, replace the node's value with its in-order predecessor (largest value in the left sub-tree) or in-order successor (smallest value in the right sub-tree).

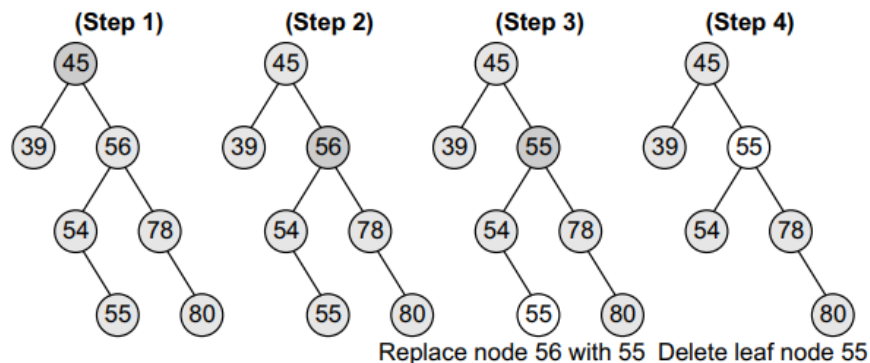


Figure: Deleting node 56 from the given binary search tree

- This deletion could also be handled by replacing node 56 with its in-order successor, as shown in Figure below.

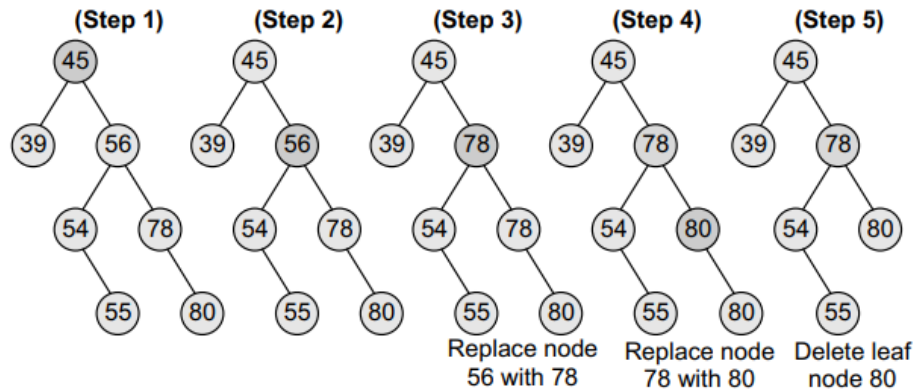


Figure: Deleting node 56 from the given binary search tree

ALGORITHM:

- In Step 1 of the algorithm, we first check if $TREE = NULL$, because if it is true, then the node to be deleted is not present in the tree. However, if that is not the case, then we check if the value to be deleted is less than the current node's data.
- In case the value is less, we call the algorithm recursively on the node's left sub-tree, otherwise the algorithm is called recursively on the node's right sub-tree.
- If we have found the node whose value is equal to VAL , then we check which case of deletion it is. If the node to be deleted has both left and right children, then we find the in-order predecessor of the node by calling $findLargestNode(TREE \rightarrow LEFT)$ and replace the current node's value with that of its in-order predecessor.
- Then, we call $Delete(TREE \rightarrow LEFT, TEMP \rightarrow DATA)$ to delete the initial node of the in-order predecessor. Thus, we reduce the case 3 of deletion into either case 1 or case 2 of deletion.
- If the node to be deleted does not have any child, then we simply set the node to $NULL$.
- Finally, if the node to be deleted has either a left or a right child but not both, then the current node is replaced by its child node and the initial child node is deleted from the tree.

```

Delete (TREE, VAL)

Step 1: IF TREE = NULL
    Write "VAL not found in the tree"
ELSE IF VAL < TREE->DATA
    Delete(TREE->LEFT, VAL)
ELSE IF VAL > TREE->DATA
    Delete(TREE->RIGHT, VAL)
ELSE IF TREE->LEFT AND TREE->RIGHT
    SET TEMP = findLargestNode(TREE->LEFT)
    SET TREE->DATA = TEMP->DATA
    Delete(TREE->LEFT, TEMP->DATA)
ELSE
    SET TEMP = TREE
    IF TREE->LEFT = NULL AND TREE->RIGHT = NULL
        SET TREE = NULL
    ELSE IF TREE->LEFT != NULL
        SET TREE = TREE->LEFT
    ELSE
        SET TREE = TREE->RIGHT
    [END OF IF]
    FREE TEMP
    [END OF IF]
Step 2: END

```

- The delete function requires time proportional to the height of the tree in the worst case. It takes $O(\log n)$ time to execute in the average case and $\Omega(n)$ time in the worst case.

NEED FOR BALANCE, ROTATION, AVL TREES

- AVL tree is a **height-balanced binary search tree**. That means, an AVL tree is also a binary search tree but it is a balanced tree.
- A binary tree is said to be balanced if, the difference between the heights of left and right subtrees of every node in the tree is either -1, 0 or +1. In other words, a binary tree is said to be balanced if the height of left and right children of every node differ by either -1, 0 or +1.
- In an AVL tree, every node maintains an extra information known as **balance factor**.
- The AVL tree was introduced in the year 1962 by G.M. Adelson-Velsky and E.M. Landis.

Balance Factor

- Balance factor of a node is the difference between the heights of the left and right subtrees of that node. The balance factor of a node is calculated either **height of left subtree - height of right subtree** (OR) **height of right subtree - height of left subtree**.

Balance factor = height of Left Subtree – height of Right Subtree

Every AVL Tree is a binary search tree but every Binary Search Tree need not be AVL tree.

Example of AVL Tree

- The below tree is a binary search tree and every node is satisfying balance factor condition. So this tree is said to be an AVL tree.

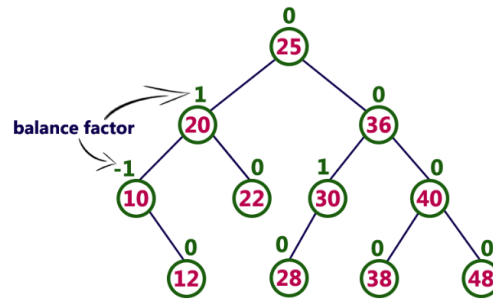


Figure: AVL Tree

AVL Tree Rotations

- In AVL tree, after performing operations like insertion and deletion we need to check the **balance factor** of every node in the tree.
- If every node satisfies the balance factor condition, then we conclude the operation otherwise we must make it balanced.
- Whenever the tree becomes imbalanced due to any operation, we use **rotation** operations to make the tree balanced. Rotation operations are used to make the tree balanced.
- There are **four** rotations and they are classified into **two** types.

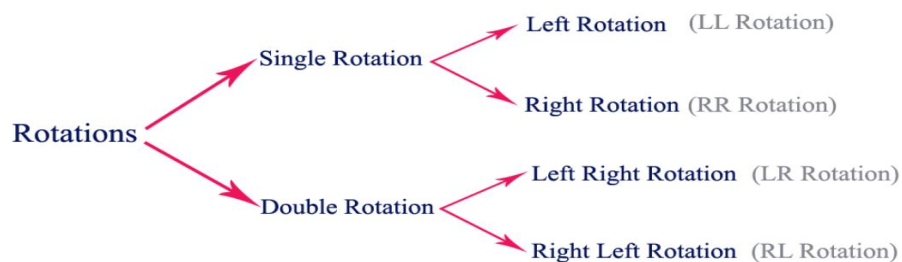
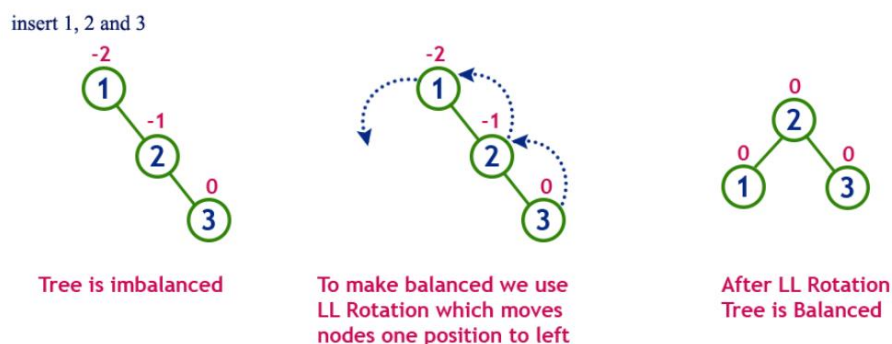


Figure: Types of Rotation

Single Left Rotation (LL Rotation)

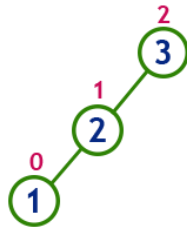
- In LL Rotation, every node moves one position to left from the current position. To understand LL Rotation, let us consider the following insertion operation in AVL Tree.



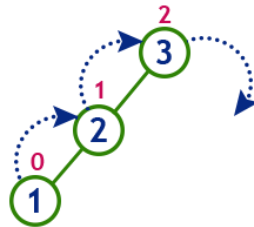
Single Right Rotation (RR Rotation)

- In RR Rotation, every node moves one position to right from the current position. To understand RR Rotation, let us consider the following insertion operation in AVL Tree...

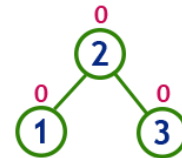
insert 3, 2 and 1



Tree is imbalanced
because node 3 has balance factor 2



To make balanced we use
RR Rotation which moves
nodes one position to right

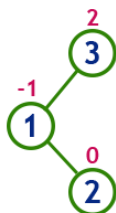


After RR Rotation
Tree is Balanced

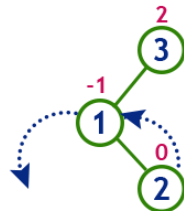
Left Right Rotation (LR Rotation)

- The LR Rotation is a sequence of single left rotation followed by a single right rotation. In LR Rotation, at first, every node moves one position to the left and one position to right from the current position. To understand LR Rotation, let us consider the following insertion operation in AVL Tree...

insert 3, 1 and 2

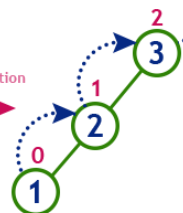


Tree is imbalanced
because node 3 has balance factor 2



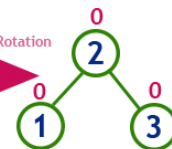
LL Rotation

After LL Rotation



RR Rotation

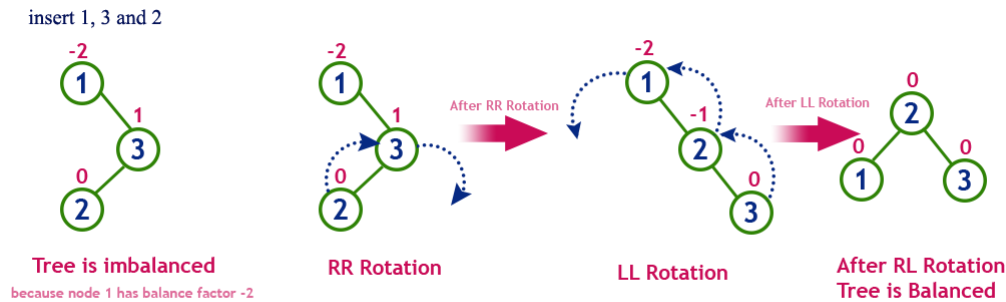
After RR Rotation



After LR Rotation
Tree is Balanced

Right Left Rotation (RL Rotation)

- The RL Rotation is sequence of single right rotation followed by single left rotation. In RL Rotation, at first every node moves one position to right and one position to left from the current position. To understand RL Rotation, let us consider the following insertion operation in AVL Tree...

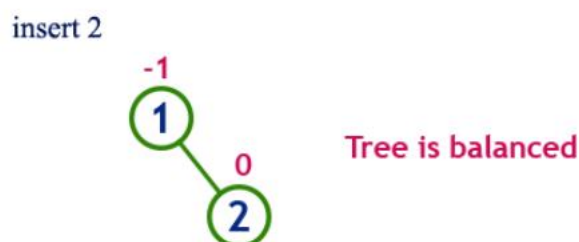
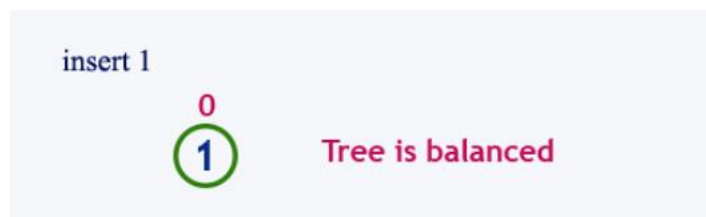


Operations on an AVL Tree

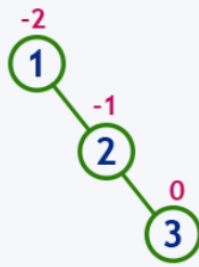
Insertion Operation in AVL Tree

- In an AVL tree, the insertion operation is performed with $O(\log n)$ time complexity. In AVL Tree, a new node is always inserted as a leaf node. The insertion operation is performed as follows...
 - Step 1 - Insert the new element into the tree using Binary Search Tree insertion logic.
 - Step 2 - After insertion, check the **Balance Factor** of every node.
 - Step 3 - If the **Balance Factor** of every node is **0 or 1 or -1** then go for next operation.
 - Step 4 - If the **Balance Factor** of any node is other than **0 or 1 or -1** then that tree is said to be imbalanced. In this case, perform suitable **Rotation** to make it balanced and go for next operation.

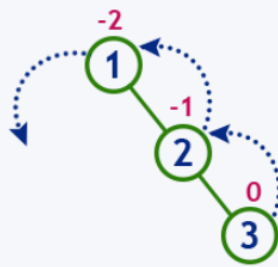
Example: Construct an AVL Tree by inserting numbers from 1 to 8.



insert 3

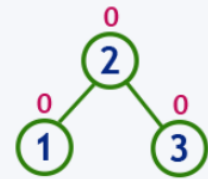


Tree is imbalanced



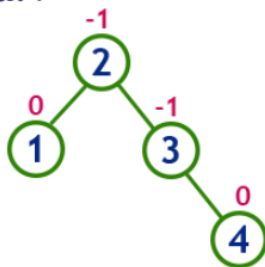
LL Rotation

After LL Rotation



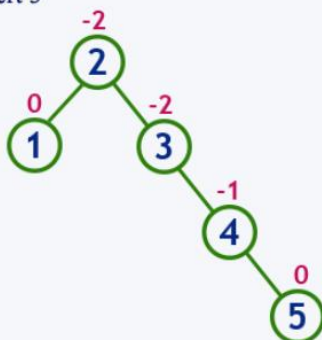
Tree is balanced

insert 4

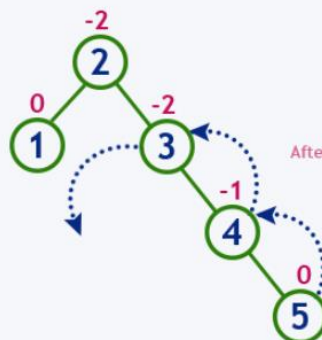


Tree is balanced

insert 5

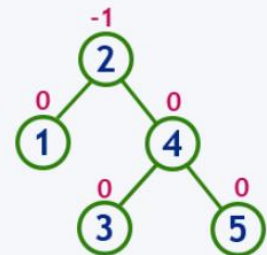


Tree is imbalanced



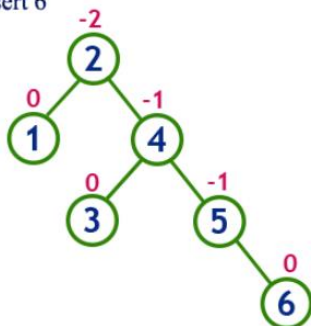
LL Rotation at 3

After LL Rotation at 3

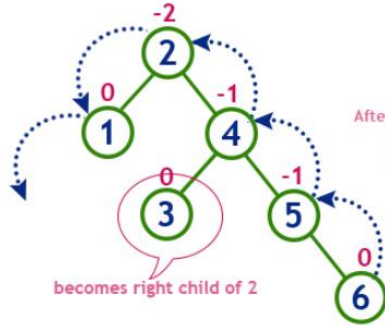


Tree is balanced

insert 6

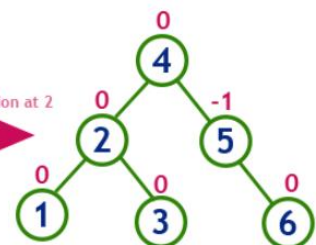


Tree is imbalanced

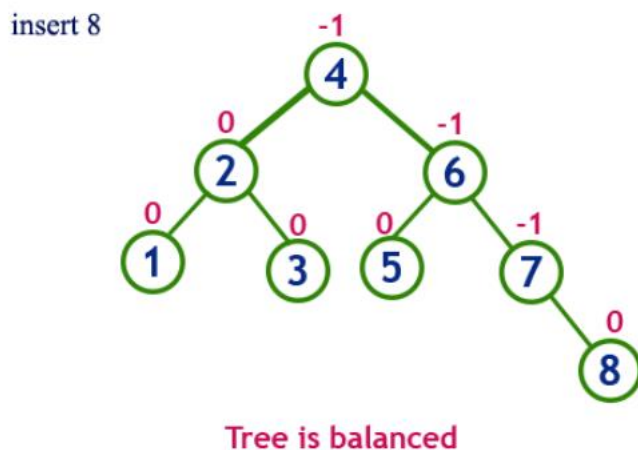
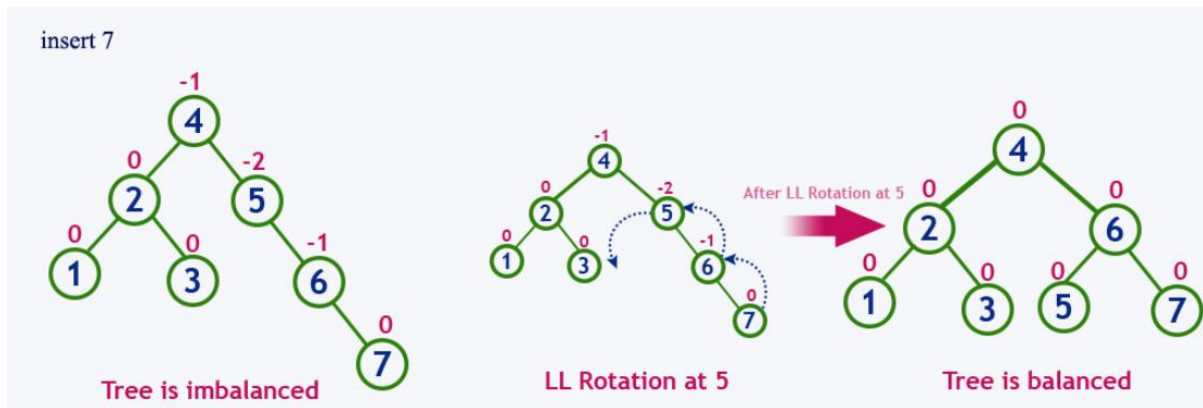


LL Rotation at 2

After LL Rotation at 2



Tree is balanced



Deletion Operation in AVL Tree

- The deletion operation in AVL Tree is similar to deletion operation in BST. But after every deletion operation, we need to check with the Balance Factor condition.
- If the tree is balanced after deletion go for next operation otherwise perform suitable rotation to make the tree Balanced.

HEAPS

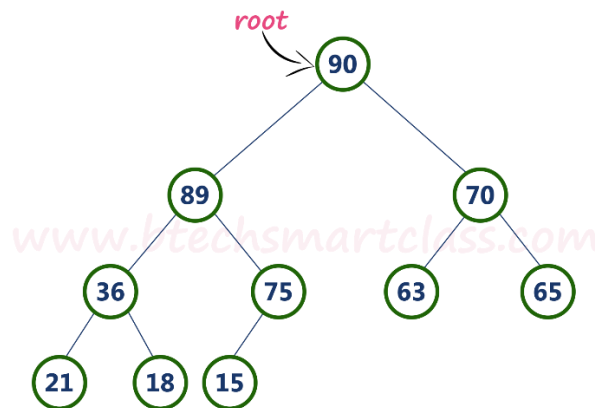
- Heap data structure is a specialized binary tree-based data structure. In a heap data structure, nodes are arranged based on their values. A heap data structure is also called as Binary Heap.
- There are two types of heap data structures and they are as follows...
 1. Max Heap
 2. Min Heap

Every heap data structure has the following properties...

- **Property #1 (Ordering):** Nodes must be arranged in an order according to their values based on Max heap or Min heap.
- **Property #2 (Structural):** All levels in a heap must be full except the last level and all nodes must be filled from left to right strictly.

Max Heap

- Max heap is a specialized full binary tree in which every parent node contains greater or equal value than its child nodes.



Above tree is satisfying both Ordering property and Structural property according to the Max Heap data structure.

Operations on Max Heap

The following operations are performed on a Max heap data structure.

1. Finding Maximum
2. Insertion
3. Deletion

Finding Maximum Value Operation in Max Heap

- Finding the node which has maximum value in a max heap is very simple. In a max heap, the root node has the maximum value than all other nodes. So, directly we can display root node value as the maximum value in max heap.

Insertion Operation in Max Heap

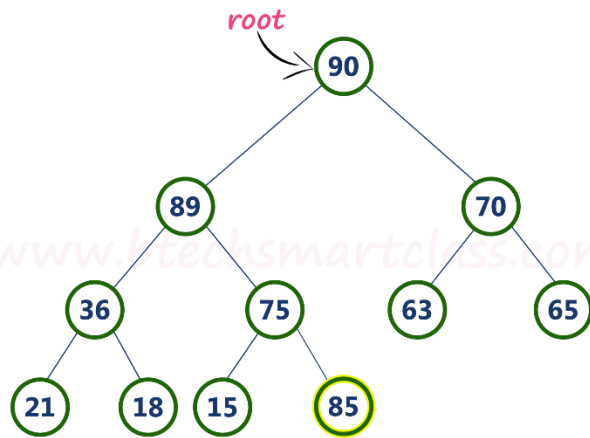
Insertion Operation in max heap is performed as follows...

- Step 1 - Insert the **newNode** as **last leaf** from left to right.
- Step 2 - Compare **newNode value** with its **Parent node**.
- Step 3 - If **newNode value is greater** than its parent, then **swap** both of them.
- Step 4 - Repeat step 2 and step 3 until newNode value is less than its parent node (or) newNode reaches to root.

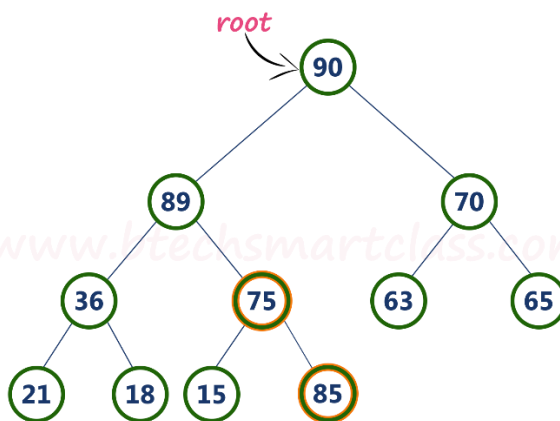
Example

Consider the above max heap. **Insert a new node with value 85.**

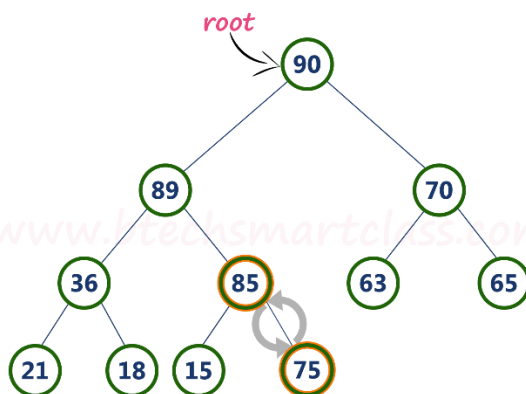
- **Step 1** - Insert the **new Node** with value 85 as **last leaf** from left to right. That means new Node is added as a right child of node with value 75. After adding max heap is as follows...



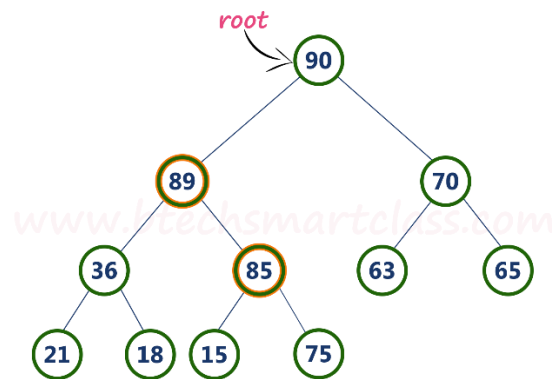
- **Step 2** - Compare **newNode value (85)** with its **Parent node value (75)**. That means **85 > 75**



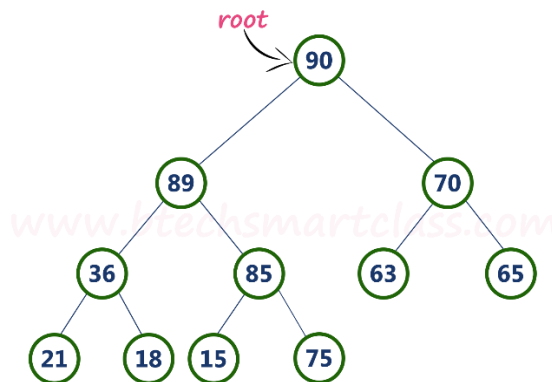
- **Step 3** - Here **newNode value (85)** is **greater than its parent value (75)**, then **swap** both of them. After swapping, max heap is as follows...



- **Step 4** - Now, again compare **newNode value (85)** with its **parent node value (89)**.



Here, **newNode value (85)** is **smaller than its parent node value (89)**. So, we stop insertion process. Finally, max heap after insertion of a new node with value 85 is as follows...



Deletion Operation in Max Heap

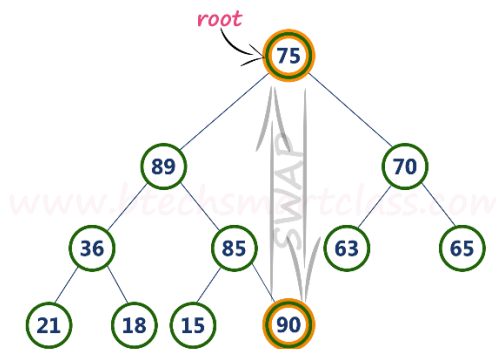
- In a max heap, deleting the last node is very simple as it does not disturb max heap properties.
- Deleting root node from a max heap is little difficult as it disturbs the max heap properties. We use the following steps to delete the root node from a max heap...
 - Step 1 - **Swap** the **root** node with **last** node in max heap
 - Step 2 - **Delete** last node.
 - Step 3 - Now, compare **root value** with its **left child value**.
 - Step 4 - If **root value is smaller** than its left child, then compare **left child** with its **right sibling**. Else goto **Step 6**
 - Step 5 - If **left child value is larger** than its **right sibling**, then **swap root** with **left child** otherwise **swap root** with its **right child**.
 - Step 6 - If **root value is larger** than its left child, then compare **root value** with its **right child** value.
 - Step 7 - If **root value is smaller** than its **right child**, then **swap root** with **right child** otherwise **stop the process**.

- **Step 8** - Repeat the same until root node fixes at its exact position.

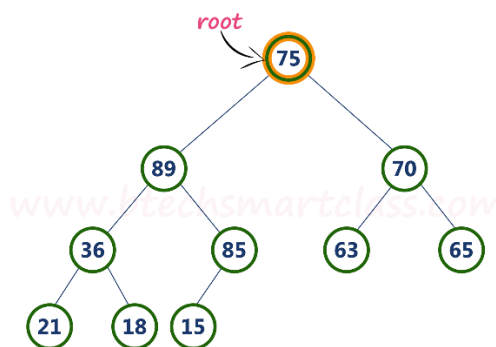
Example

Consider the above max heap. Delete root node (90) from the max heap.

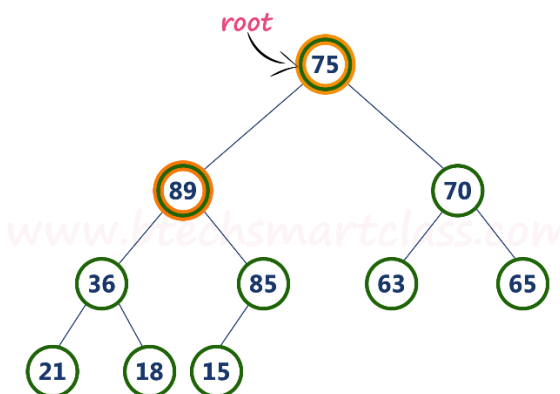
- **Step 1** - Swap the root node (90) with last node 75 in max heap. After swapping max heap is as follows...



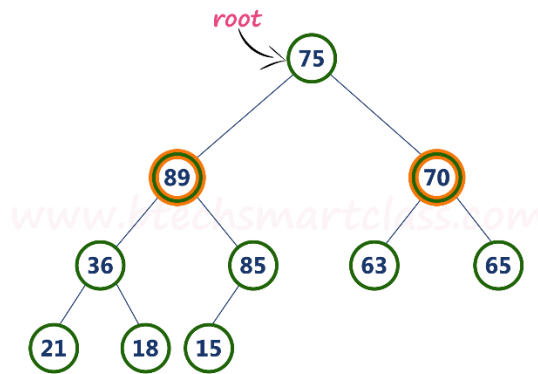
- **Step 2** - Delete last node. Here the last node is 90. After deleting node with value 90 from heap, max heap is as follows...



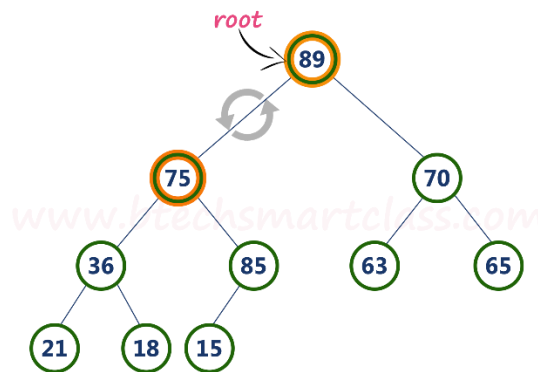
- **Step 3** - Compare root node (75) with its left child (89).



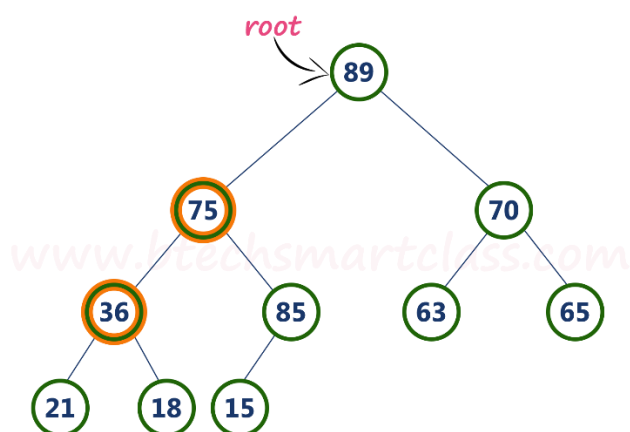
Here, **root value (75)** is smaller than its left child value (89). So, compare left child (89) with its right sibling (70).



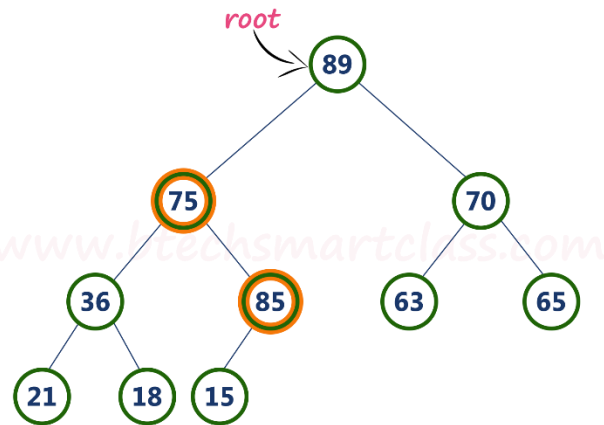
- **Step 4** - Here, **left child value (89)** is larger than its **right sibling (70)**, So, swap **root (75)** with left child (89).



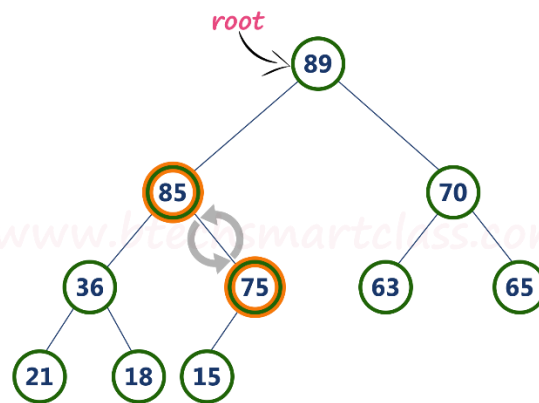
- **Step 5** - Now, again **compare 75** with its **left child (36)**.



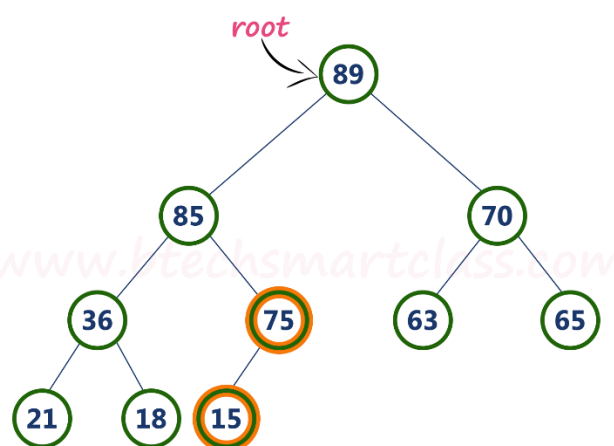
Here, node with **value 75** is larger than its left child. So, we compare **node 75** with its **right child 85**.



- **Step 6** - Here, node with **value 75** is smaller than its **right child (85)**. So, we swap both of them. After swapping max heap is as follows...

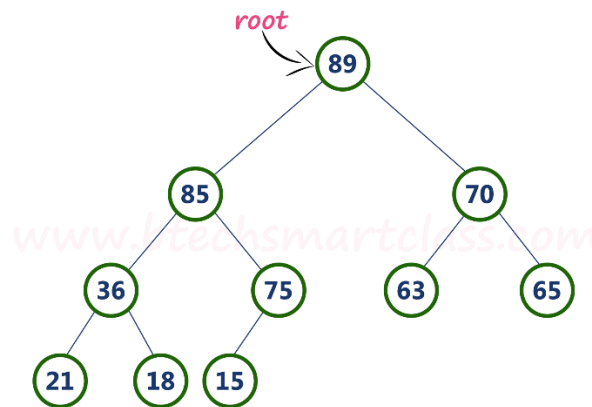


- **Step 7** - Now, compare node with **value 75** with its **left child (15)**.



- Here, node with value **75** is larger than its left child (**15**) and it does not have right child. So we stop the process.

- Finally, **max heap** after **deleting root node (90)** is as follows...



Min Heap

- The value of the **root node** is **less than or equal** to either of its children.

HASH FUNCTIONS - INTRODUCTION, FUNCTIONS, COLLISION AVOIDANCE, SEPARATE CHAINING, OPEN ADDRESSING, LINEAR PROBING, QUADRATIC PROBING.

HASH TABLE, HASHING

- Hash table** is a data structure in which keys are mapped to array positions by a hash function.
- A value stored in a hash table can be searched in **O(1) time** by using a hash function which generates an address from the key (by producing the index of the array where the value is stored).
- In a hash table, an element with **key k** is stored at **index h(k)** and not k. It means a **hash function h** is used to calculate the index at which the element with key k will be stored.
- This process of mapping the keys to appropriate locations (or indices) in a hash table is called **hashing**.

HASH FUNCTION

- A **hash function** is a **mathematical formula** which, when applied to a key, produces an integer which can be used as an index for the key in the hash table.
- The **main aim of a hash function** is that elements should be relatively, randomly, and uniformly distributed. It produces a unique set of integers within some suitable range in order to reduce the number of collisions.
- One of the mostly used hash function is **DIVISION METHOD**.

Division Method

- It is the simplest method of hashing an **integer x**. This method **divides x by M** and then uses the remainder obtained. In this case, the hash function can be given as

$$h(x) = x \bmod M$$

- The division method is quite good for just about any value of M and since it requires only a single division operation, the method works very fast. However, extra care should be taken to select a suitable value for **M**.

Example:

Calculate the hash values of keys 1234 and 5462 using Division Method

Solution :

Setting M = 97, hash values can be calculated as:

$$h(1234) = 1234 \% 97 = 70$$

$$h(5642) = 5642 \% 97 = 16$$

COLLISION

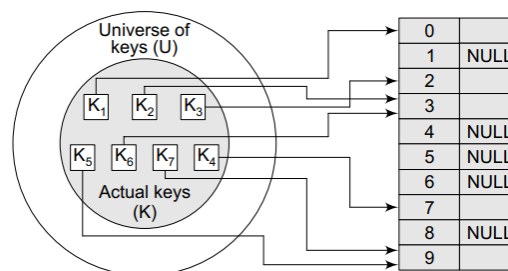


Figure: Relationship between keys and hash table index

- Figure above shows a **hash table** in which each key from the set K is mapped to locations generated by using a **hash function**.
- Note that **keys k2 and k6 point to the same memory location**. This is known as **collision**. That is, **when two or more keys map to the same memory location, a collision is said to occur**.
- Similarly, keys k5 and k7 also collide.

COLLISION RESOLUTION TECHNIQUE

- Collisions occur when the hash function maps two different keys to the same location**. Obviously, two records cannot be stored in the same location. Therefore, a method used **to solve the problem of collision, also called collision resolution technique**, is applied.
- The two most popular methods of resolving collisions are:
 - Open addressing
 - Separate Chaining

COLLISION RESOLUTION BY OPEN ADDRESSING

- Once a **collision** takes place, **open addressing or closed hashing** computes new positions using a probe sequence and the next record is stored in that position.
- In this technique, all the values are stored in the hash table. The **hash table** contains two types of values: **sentinel values (e.g., -1) and data values**.
- The presence of a sentinel value indicates that the location contains no data value at present but can be used to hold a value.
- The process of examining memory locations in the hash table is called **probing**.
- **Open addressing technique** can be implemented using
 - **linear probing**
 - **quadratic probing**
 - **double hashing**

Linear Probing

- The simplest approach to resolve a collision is linear probing. In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + i] \bmod m$$

- Where m is the size of the hash table, $h'(k) = (k \bmod m)$, and i is the probe number that varies from 0 to $m-1$

Example:

Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table.

Let $h'(k) = k \bmod m$, $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step 1

Key = 72

$$\begin{aligned}h(72, 0) &= (72 \bmod 10 + 0) \bmod 10 \\&= (2) \bmod 10 \\&= 2\end{aligned}$$

Since $\tau[2]$ is vacant, insert key 72 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2

Key = 27

$$\begin{aligned}h(27, 0) &= (27 \bmod 10 + 0) \bmod 10 \\&= (7) \bmod 10 \\&= 7\end{aligned}$$

Since $\tau[7]$ is vacant, insert key 27 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3

Key = 36

$$\begin{aligned}
 h(36, 0) &= (36 \bmod 10 + 0) \bmod 10 \\
 &= (6) \bmod 10 \\
 &= 6
 \end{aligned}$$

Since $\tau[6]$ is vacant, insert key 36 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4

Key = 24

$$h(24, 0) = (24 \bmod 10 + 0) \bmod 10$$

$$\begin{aligned}
 &= (4) \bmod 10 \\
 &= 4
 \end{aligned}$$

Since $\tau[4]$ is vacant, insert key 24 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5

Key = 63

$$\begin{aligned}
 h(63, 0) &= (63 \bmod 10 + 0) \bmod 10 \\
 &= (3) \bmod 10 \\
 &= 3
 \end{aligned}$$

Since $\tau[3]$ is vacant, insert key 63 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6

Key = 81

$$\begin{aligned}
 h(81, 0) &= (81 \bmod 10 + 0) \bmod 10 \\
 &= (1) \bmod 10 \\
 &= 1
 \end{aligned}$$

Since $\tau[1]$ is vacant, insert key 81 at this location.

0	1	2	3	4	5	6	7	8	9
0	81	72	63	24	-1	36	27	-1	-1

Step 7

Key = 92

$$\begin{aligned}
 h(92, 0) &= (92 \bmod 10 + 0) \bmod 10 \\
 &= (2) \bmod 10 \\
 &= 2
 \end{aligned}$$

Now $\tau[2]$ is occupied, so we cannot store the key 92 in $\tau[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$\begin{aligned}
 h(92, 1) &= (92 \bmod 10 + 1) \bmod 10 \\
 &= (2 + 1) \bmod 10 \\
 &= 3
 \end{aligned}$$

Now $\tau[3]$ is occupied, so we cannot store the key 92 in $\tau[3]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$\begin{aligned}
 h(92, 2) &= (92 \bmod 10 + 2) \bmod 10 \\
 &= (2 + 2) \bmod 10 \\
 &= 4
 \end{aligned}$$

Now $\tau[4]$ is occupied, so we cannot store the key 92 in $\tau[4]$. Therefore, try again for the next location. Thus probe, $i = 3$, this time.

Key = 92

$$\begin{aligned}
 h(92, 3) &= (92 \bmod 10 + 3) \bmod 10 \\
 &= (2 + 3) \bmod 10 \\
 &= 5
 \end{aligned}$$

Since $\tau[5]$ is vacant, insert key 92 at this location.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

Step 8

$$\begin{aligned}\text{Key} &= 101 \\ h(101, 0) &= (101 \bmod 10 + 0) \bmod 10 \\ &= (1) \bmod 10 \\ &= 1\end{aligned}$$

Now $\tau[1]$ is occupied, so we cannot store the key 101 in $\tau[1]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

$$\begin{aligned}\text{Key} &= 101 \\ h(101, 1) &= (101 \bmod 10 + 1) \bmod 10 \\ &= (1 + 1) \bmod 10 \\ &= 2\end{aligned}$$

$\tau[2]$ is also occupied, so we cannot store the key in this location. The procedure will be repeated until the hash function generates the address of location 8 which is vacant and can be used to store the value in it.

Quadratic Probing

- In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

where m is the size of the hash table, $h'(k) = (k \bmod m)$, i is the probe number that varies from 0 to $m-1$, and c_1 and c_2 are constants such that c_1 and $c_2 \neq 0$.

- Quadratic probing eliminates the primary clustering phenomenon of linear probing because instead of doing a linear search, it does a quadratic search. For a given key k , first the location generated by $h'(k) \bmod m$ is probed.
- If the location is free, the value is stored in it, else subsequent locations probed are offset by factors that depend in a quadratic manner on the probe number i .
- Although quadratic probing performs better than linear probing, in order to maximize the utilization of the hash table, the values of c_1 , c_2 , and m need to be constrained.

Example:

Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$.

Solution:

Let $h'(k) = k \bmod m$, $m = 10$ Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

We have,

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

Step 1

$$\begin{aligned}\text{Key} &= 72 \\ h(72, 0) &= [72 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\ &= [72 \bmod 10] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2\end{aligned}$$

Since $\tau[2]$ is vacant, insert the key 72 in $\tau[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 Key = 27

$$\begin{aligned}
h(27, 0) &= [27 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
&= [27 \bmod 10] \bmod 10 \\
&= 7 \bmod 10 \\
&= 7
\end{aligned}$$

Since $\tau[7]$ is vacant, insert the key 27 in $\tau[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned}
h(36, 0) &= [36 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
&= [36 \bmod 10] \bmod 10 \\
&= 6 \bmod 10 \\
&= 6
\end{aligned}$$

Since $\tau[6]$ is vacant, insert the key 36 in $\tau[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$\begin{aligned}
h(24, 0) &= [24 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
&= [24 \bmod 10] \bmod 10 \\
&= 4 \bmod 10 \\
&= 4
\end{aligned}$$

Since $\tau[4]$ is vacant, insert the key 24 in $\tau[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5 Key = 63

$$\begin{aligned}
h(63, 0) &= [63 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
&= [63 \bmod 10] \bmod 10 \\
&= 3 \bmod 10 \\
&= 3
\end{aligned}$$

Since $\tau[3]$ is vacant, insert the key 63 in $\tau[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 Key = 81

$$\begin{aligned}
h(81, 0) &= [81 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
&= [81 \bmod 10] \bmod 10 \\
&= 1 \bmod 10 \\
&= 1
\end{aligned}$$

Since $\tau[1]$ is vacant, insert the key 81 in $\tau[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7 Key = 101

$$\begin{aligned} h(101,0) &= [101 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\ &= [101 \bmod 10 + 0] \bmod 10 \\ &= 1 \bmod 10 \\ &= 1 \end{aligned}$$

Since $\tau[1]$ is already occupied, the key 101 cannot be stored in $\tau[1]$. Therefore, try again for next location. Thus probe, $i = 1$, this time.

```
Key = 101
h(101,0) = [101 mod 10 + 1 × 1 + 3 × 1] mod 10
          = [101 mod 10 + 1 + 3] mod 10
          = [101 mod 10 + 4] mod 10
          = [1 + 4] mod 10
          = 5 mod 10
          = 5
```

Since $\tau[5]$ is vacant, insert the key 101 in $\tau[5]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	101	36	27	-1	-1

Double Hashing

- To start with, double hashing uses one hash value and then repeatedly steps forward an interval until an empty location is reached.
- The interval is decided using a second, independent hash function, hence the name double hashing.
- In double hashing, we use two hash functions rather than a single function. The hash function in the case of double hashing can be given as:

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

where m is the size of the hash table, $h_1(k)$ and $h_2(k)$ are two hash functions given as $h_1(k) = k \bmod m$, $h_2(k) = k \bmod m'$, i is the probe number that varies from 0 to $m-1$, and m' is chosen to be less than m . We can choose $m' = m-1$ or $m-2$.

Pros and Cons

- Double hashing minimizes repeated collisions and the effects of clustering. That is, double hashing is free from problems associated with primary clustering as well as secondary clustering.

Example:

Consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Solution

Let $m = 10$ Initially, the hash table can be given as:

[illegible]

We have,

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

Step 1

Key = 72

$$\begin{aligned} h(72, 0) &= [72 \bmod 10 + (0 \times 72 \bmod 8)] \bmod 10 \\ &= [2 + (0 \times 0)] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2 \end{aligned}$$

Since $\tau[2]$ is vacant, insert the key 72 in $\tau[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2

Key = 27

$$\begin{aligned} h(27, 0) &= [27 \bmod 10 + (0 \times 27 \bmod 8)] \bmod 10 \\ &= [7 + (0 \times 3)] \bmod 10 \\ &= 7 \bmod 10 \\ &= 7 \end{aligned}$$

Since $\tau[7]$ is vacant, insert the key 27 in $\tau[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3

Key = 36

$$\begin{aligned} h(36, 0) &= [36 \bmod 10 + (0 \times 36 \bmod 8)] \bmod 10 \\ &= [6 + (0 \times 4)] \bmod 10 \\ &= 6 \bmod 10 \\ &= 6 \end{aligned}$$

Since $\tau[6]$ is vacant, insert the key 36 in $\tau[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4

Key = 24

$$\begin{aligned} h(24, 0) &= [24 \bmod 10 + (0 \times 24 \bmod 8)] \bmod 10 \\ &= [4 + (0 \times 0)] \bmod 10 \\ &= 4 \bmod 10 \\ &= 4 \end{aligned}$$

Since $\tau[4]$ is vacant, insert the key 24 in $\tau[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5

Key = 63

$$\begin{aligned} h(63, 0) &= [63 \bmod 10 + (0 \times 63 \bmod 8)] \bmod 10 \\ &= [3 + (0 \times 7)] \bmod 10 \\ &= 3 \bmod 10 \\ &= 3 \end{aligned}$$

Since $\tau[3]$ is vacant, insert the key 63 in $\tau[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6

Key = 81

$$\begin{aligned}
 h(81, 0) &= [81 \bmod 10 + (0 \times 81 \bmod 8)] \bmod 10 \\
 &= [1 + (0 \times 1)] \bmod 10 \\
 &= 1 \bmod 10 \\
 &= 1
 \end{aligned}$$

Since $\tau[1]$ is vacant, insert the key 81 in $\tau[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7

Key = 92

$$\begin{aligned}
 h(92, 0) &= [92 \bmod 10 + (0 \times 92 \bmod 8)] \bmod 10 \\
 &= [2 + (0 \times 4)] \bmod 10 \\
 &= 2 \bmod 10 \\
 &= 2
 \end{aligned}$$

Now $\tau[2]$ is occupied, so we cannot store the key 92 in $\tau[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$h(92, 1) = [92 \bmod 10 + (1 \times 92 \bmod 8)] \bmod 10$$

$$\begin{aligned}
 &= [2 + (1 \times 4)] \bmod 10 \\
 &= (2 + 4) \bmod 10 \\
 &= 6 \bmod 10 \\
 &= 6
 \end{aligned}$$

Now $\tau[6]$ is occupied, so we cannot store the key 92 in $\tau[6]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$\begin{aligned}
 h(92, 2) &= [92 \bmod 10 + (2 \times 92 \bmod 8)] \bmod 10 \\
 &= [2 + (2 \times 4)] \bmod 10 \\
 &= [2 + 8] \bmod 10 \\
 &= 10 \bmod 10 \\
 &= 0
 \end{aligned}$$

Since $\tau[0]$ is vacant, insert the key 92 in $\tau[0]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Step 8

Key = 101

$$\begin{aligned}
 h(101, 0) &= [101 \bmod 10 + (0 \times 101 \bmod 8)] \bmod 10 \\
 &= [1 + (0 \times 5)] \bmod 10 \\
 &= 1 \bmod 10 \\
 &= 1
 \end{aligned}$$

Now $\tau[1]$ is occupied, so we cannot store the key 101 in $\tau[1]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 101

$$\begin{aligned}
 h(101, 1) &= [101 \bmod 10 + (1 \times 101 \bmod 8)] \bmod 10 \\
 &= [1 + (1 \times 5)] \bmod 10 \\
 &= [1 + 5] \bmod 10 \\
 &= 6
 \end{aligned}$$

COLLISION RESOLUTION BY SEPARATE CHAINING

- The idea behind separate chaining is to implement the array as a linked list called a chain. The **linked list** data structure is used to implement this technique.
- When multiple elements are hashed into the same slot index, then these elements are inserted into a singly-linked list which is known as a chain.

Example:

Let us consider a simple hash function as “**key mod 5**” and a sequence of keys as 12, 22, 15, 25

